

Classifying Consumer Financial Complaints by Product Category: A Comparative Study of Text Representations and Neural Architectures.

Nabil Ishmam

24241245

BRAC University

nabil.ishmam@g.bracu.ac.bd

Quazi Unjurn Daniel

23201133

BRAC University

quazi.unjurn.daniel@g.bracu.ac.bd

Shoumodip Paul

23201447

BRAC University

shoumodip.paul@g.bracu.ac.bd

Afnan Mojumder

23301519

BRAC University

afnan.mojumder@g.bracu.ac.bd

Abstract

We classify 2.02 million U.S. Consumer Financial Protection Bureau (CFPB) complaint narratives into nine product categories, comparing ten architectures spanning three paradigms: classical bag-of-words models over TF-IDF, six recurrent networks over pre-trained Word2Vec embeddings, and a fine-tuned BERT Base transformer. All ten models are trained under an identical 200,000-document budget and scored on an identical 303,213-document test split, which makes the resulting ranking attributable to architecture rather than to training-set size. BERT Base achieves the best test Macro F1 (0.7647), but Logistic Regression reaches 0.7367 while running roughly 1,300 times faster at inference. Our novelty contribution is a controlled study of imbalance handling on a corpus with a 52.9:1 majority-to-minority ratio. Across thirteen controlled runs we find that the standard remedy is frequently harmful: for Logistic Regression, Naive Bayes and Bidirectional LSTM the best strategy for minority-class F1 is *no mitigation at all*, and inverse-frequency class weighting costs the Bidirectional LSTM 7.9 F1 points on the four rarest classes. Mitigation helps decisively only for Random Forest, the one architecture that collapses without it. We also show that masking padded positions in the recurrent models raises SimpleRNN Macro F1 from 0.2222 to 0.6121, indicating that a widely reported “vanishing-gradient” failure of simple recurrence on this task is partly an artefact of reading hidden states off padding.

1 Introduction

The CFPB publishes consumer complaints about financial products, and each complaint must be

routed to the responsible product team. Manual triage does not scale to the roughly two million narratives now in the public database ([Consumer Financial Protection Bureau, 2024](#)), and misrouting delays regulatory response on exactly the products where consumers have least recourse. Automatic classification of the free-text narrative into a product category is therefore a concrete operational problem with a public, well-documented dataset.

The task is a nine-way, single-label text classification problem. Its defining property is severe class imbalance: *Credit reporting* accounts for 59.60% of the corpus while *Payday / title / personal loan* accounts for 1.13%, a ratio of 52.9:1. Any evaluation that leans on accuracy is therefore uninformative, and any comparison that does not control for imbalance handling risks measuring the mitigation rather than the model.

This work addresses three research questions.

RQ1 Under an identical training budget and an identical test split, how do classical, recurrent and transformer architectures actually compare on this task, and what does the accuracy gain cost in compute?

RQ2 Does the choice of imbalance-handling strategy—class weighting, SMOTE oversampling, or focal loss—depend on the architecture it is applied to?

RQ3 How much of the reported weakness of simple recurrent models on long documents is attributable to sequence-padding artefacts rather than to the recurrence itself?

Our contributions are: (i) a budget-controlled

comparison of ten architectures in which every model sees the same data, removing the confound that makes many published comparisons uninterpretable; (ii) thirteen controlled imbalance experiments showing that mitigation is usually harmful on this corpus and helps only the architecture that fails without it; and (iii) evidence that padding masking, not gating, accounts for most of the gap between SimpleRNN and gated recurrent cells here.

2 Related Work

Text classification baselines. TF-IDF term weighting (Spärck Jones, 1972) with a linear classifier remains a strong baseline for topical text classification (Joachims, 1998), and multinomial Naive Bayes is competitive at a fraction of the cost (McCallum and Nigam, 1998). Random forests (Breiman, 2001) extend to sparse text but are sensitive to skewed priors, as our results confirm.

Recurrent and contextual models. Simple recurrent networks (Elman, 1990) suffer from vanishing gradients over long sequences, which motivated gated cells—LSTM (Hochreiter and Schmidhuber, 1997) and GRU (Cho et al., 2014)—and bidirectional processing (Schuster and Paliwal, 1997). Distributed word representations (Mikolov et al., 2013; Pennington et al., 2014) supply the input embeddings. BERT (Devlin et al., 2019) replaced recurrence with pre-trained bidirectional self-attention and defines the current strong baseline for sentence- and document-level classification.

Class imbalance. The imbalance problem is long studied (Japkowicz and Stephen, 2002), and modern surveys catalogue the standard remedies (Johnson and Khoshgoftaar, 2019): reweighting the loss by inverse class frequency, resampling the training distribution—most prominently SMOTE (Chawla et al., 2002)—and reshaping the loss so that easy majority examples contribute less gradient, as in focal loss (Lin et al., 2017). These techniques are usually reported as improvements. Our results indicate that on a corpus of this size the picture is considerably more conditional, and that reporting recall alone conceals a substantial precision cost.

Positioning. Public implementations of CFPB complaint classification that we surveyed apply class weighting as a default and do not compare it against alternatives on a common budget. To

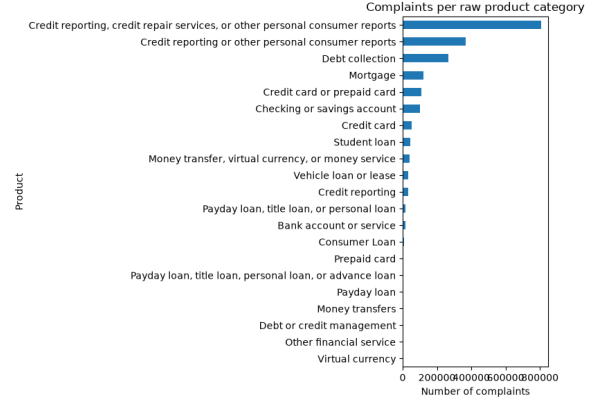


Figure 1: Raw *Product* field, 21 categories. The top two bars are both credit-reporting labels from different revisions of the CFPB taxonomy, and *Credit reporting* appears again lower down; treating them as distinct classes would fragment the training signal for a single underlying category.

our knowledge, the controlled comparison in Section 4.5—five architectures against up to four strategies, all on identical data and scored on the identical full test split—has not previously been reported for this dataset. We restrict this claim to the implementations we examined; it is not a statement about all prior work.

3 Methodology

3.1 Dataset and Exploratory Analysis

We use the public CFPB consumer complaint corpus distributed via Kaggle (Abbasov, 2024; Consumer Financial Protection Bureau, 2024). The raw file contains 2,023,066 rows across 11 columns (2.5 GB in memory). Every row carries narrative text, so narrative coverage is 100%, and there are **zero duplicate rows**. Missing values are confined to metadata fields not used for modelling (*Sub-issue* 230,559; *Sub-product* 52,206; *State* 7,344); the narrative and target columns have none.

Label consolidation. The raw *Product* field has 21 categories with substantial redundancy (three separate credit-reporting variants, two payday-loan variants). We consolidate these into nine coherent classes, dropping 1,196 rows with no clean mapping. After preprocessing removes 450 narratives that become empty, **2,021,420** documents remain. Figure 1 shows the raw label space before consolidation; the three credit-reporting variants are visible as separate bars.

Table 1 gives the resulting distribution, visualised in Figure 2. Narrative length is heavily right-

Class	Count	%
Credit reporting	1,205,275	59.60
Debt collection	266,842	13.20
Credit card / prepaid card	163,710	8.10
Mortgage	119,116	5.89
Bank account or service	115,332	5.71
Student loan	44,241	2.19
Money transfer / virtual cur.	43,016	2.13
Vehicle / consumer loan	41,538	2.05
Payday / title / personal loan	22,800	1.13

Table 1: Nine-class distribution after consolidation. The majority-to-minority ratio is 52.9:1.

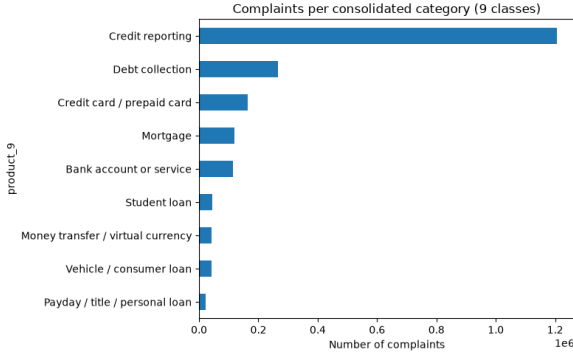


Figure 2: Class distribution after consolidation into nine categories.

skewed: mean 181.5 words, median 119, standard deviation 232.5, interquartile range 62–215, maximum 6,314 (Figure 3). This skew motivates our choice of a 256-token window and, as Section 4.2 shows, makes padding handling consequential.

Vocabulary overlap. Figure 4 shows per-class word clouds over the cleaned narratives. The overlaps are directly predictive of the errors reported in Section 4: *Bank account* and *Money transfer* share *account*, *bank*, *money* and even the same institution names, while *Credit reporting* and *Debt collection* share *credit report* and *consumer report*. *Student loan* is lexically distinctive because servicer names (*mohela*, *nelnet*, *navient*) appear nowhere else.

3.2 Preprocessing

Because the two model families require different input properties, we build **two preprocessing paths** from the same source text rather than forcing a single compromise pipeline.

Contextual path (for BERT): we strip URLs, e-mail addresses and the CFPB’s XXXX PII redaction markers, then normalise whitespace and repeated punctuation. Casing, punctuation and sentence structure are **preserved**, because BERT’s WordPiece vocabulary and positional encoding exploit

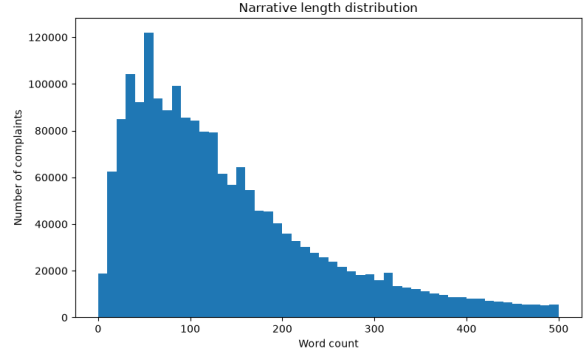


Figure 3: Narrative word-count distribution. The long right tail motivates the 256-token truncation window.

them.

Classical path (for TF-IDF and Word2Vec): we additionally lowercase, expand contractions, remove non-alphabetic characters, drop English stopwords and discard tokens of two characters or fewer.

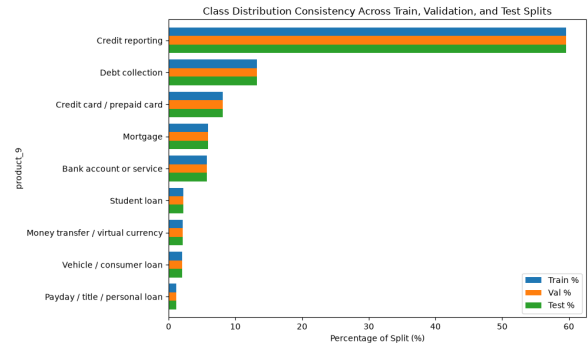
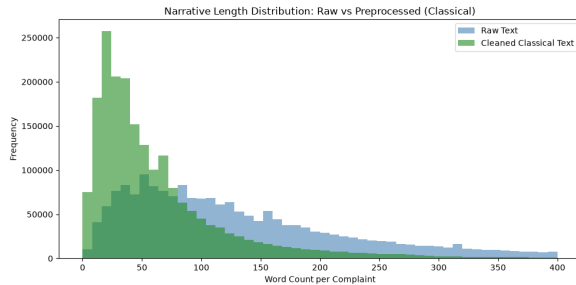
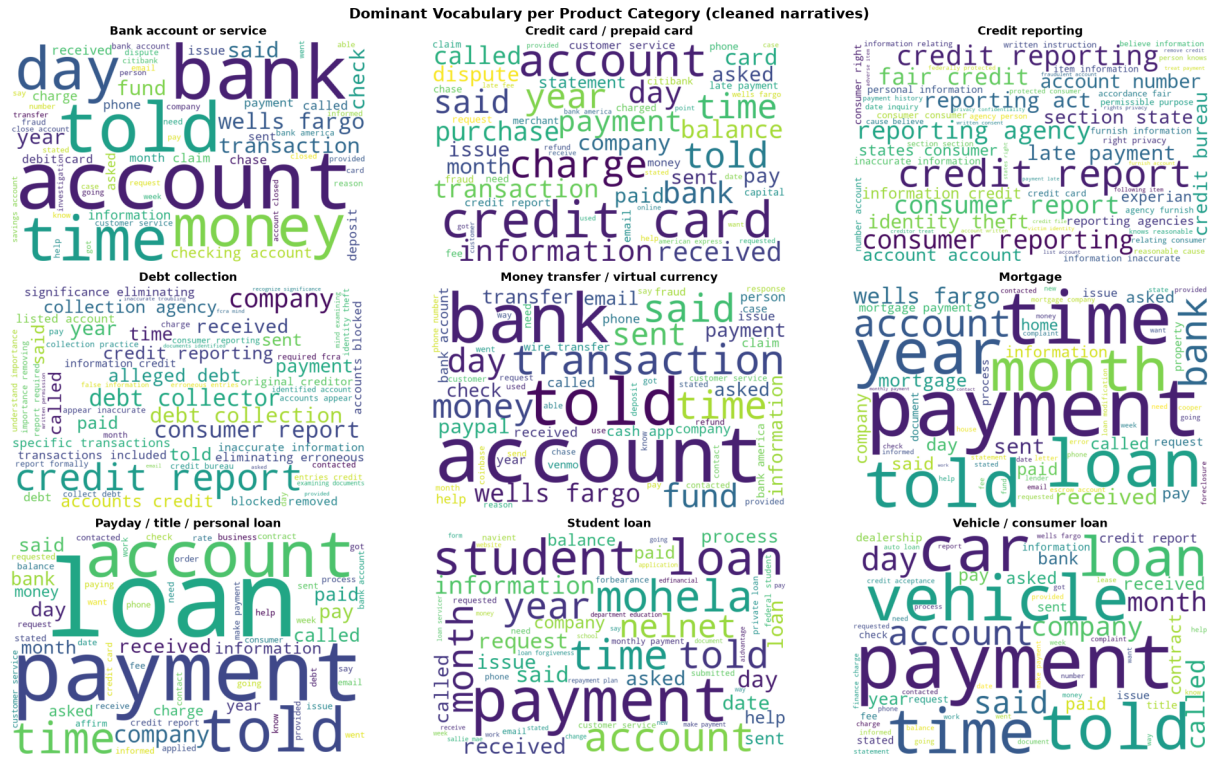
The justification is empirical. The classical path reduces the median document from 119 words to **44** and the mean from 181.5 to 70.0 (Figure 5), a 61% reduction in tokens that must be carried through a recurrent hidden state. Applying that aggressive filtering to the BERT input would discard precisely the syntax its pre-training relies on; applying the conservative treatment to TF-IDF would inflate the feature space with stopwords carrying no product signal. We also verify that no non-English narratives require handling: zero records fall below 80% ASCII content.

3.3 Train/Validation/Test Split

We split 70/15/15 into 1,414,994 training, 303,213 validation and 303,213 test documents. The split is **stratified** on the nine-class label and seeded (`random_state=42`), making it exactly reproducible. We verify stratification empirically rather than assuming it: *Credit reporting* occupies 59.60% of all three partitions and *Payday loan* 1.13% of all three (Figure 6). All fitting—the TF-IDF vocabulary, the Word2Vec embeddings and every classifier—uses training data only; validation and test are transform-only, so no target leakage is possible.

3.4 Word Representations

We implement three representations, satisfying the requirement of at least two of TF-IDF, Word2Vec and GloVe.



TF-IDF (Spärck Jones, 1972): 25,000 features over unigrams and bigrams, sublinear term-frequency scaling, min_df=5, max_df=0.85. Fitted on the full training split, giving a $1,414,994 \times 25,000$ sparse matrix with 103,726,073 non-zero entries (0.29% density). The highest mean-weight features are *credit*, *account*, *report*, *information*.

Word2Vec (Mikolov et al., 2013): 100-dimensional CBOW embeddings trained in-domain on the full training corpus for five epochs (min_count=5), via Gensim (Řehůřek and Sojka, 2010). In-domain training is preferable to pre-trained GloVe vectors here because financial-

complaint vocabulary (*escrow*, *chargeback*, *servicer* names) is poorly covered by general-corpus embeddings. These vectors initialise the recurrent models’ embedding layer, which is then fine-tuned.

WordPiece: BERT’s own subword vocabulary, used for the transformer.

3.5 Model Architectures and Protocol

Fixed training budget. Fine-tuning BERT over 1.41M documents is not tractable on a single consumer GPU within this project’s timeframe, so the transformer sets the ceiling for the whole com-

parison. We therefore train **every** architecture on the same stratified 200,000-document subsample and score every architecture on the **full** 303,213-document test split. Representations are still fitted on the full 1.41M training split; only the classifier fitting budget is capped. This is the central methodological decision of the study: without it, a ranking would confound architecture with training-set size.

Classical models. Logistic Regression (lbfgs, max_iter=1000), Multinomial Naive Bayes, and Random Forest (50 trees), via scikit-learn (Pedregosa et al., 2011).

Recurrent models. A single configurable module implements SimpleRNN, GRU and LSTM in unidirectional and bidirectional form (six architectures), in PyTorch (Paszke et al., 2019). Inputs are 256-token padded sequences initialised with the Word2Vec matrix. Crucially, the final hidden state is read through pack_padded_sequence, so each sequence contributes its true final token rather than a state advanced through padding (Section 4.2). Training uses Adam, gradient-norm clipping at 1.0, and four epochs, with the reported epoch selected on validation Macro F1.

Transformer. bert-base-uncased (Devlin et al., 2019) via Hugging Face Transformers (Wolf et al., 2020), fine-tuned for two epochs with AdamW, batch size 32, and bfloat16 mixed precision.

Hyperparameter tuning. Every model is tuned over at least three configurations against validation Macro F1, for **30 tuning runs** in total: nine classical (three each), eighteen recurrent (three configurations $\times$ six architectures) and three BERT learning rates $\{2, 3, 5\} \times 10^{-5}$. All runs are recorded in the notebook’s Section 3.8 table and summarised in Figure 7. BERT’s optimum is the lowest learning rate (2×10^{-5} , 0.7647); the two higher rates are indistinguishable from each other (0.7480 and 0.7499).

4 Results

4.1 Model Comparison

Table 2 reports the main comparison, visualised in Figure 8. **BERT Base is the best-performing model** (Macro F1 0.7647) and **SimpleRNN the worst** (0.5685), a spread of 19.62 points.

Three findings deserve emphasis.

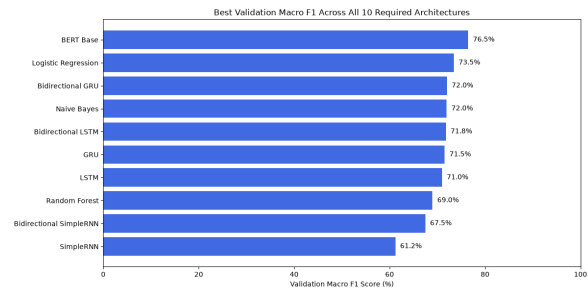


Figure 7: Best validation Macro F1 per architecture across all 30 tuning runs.

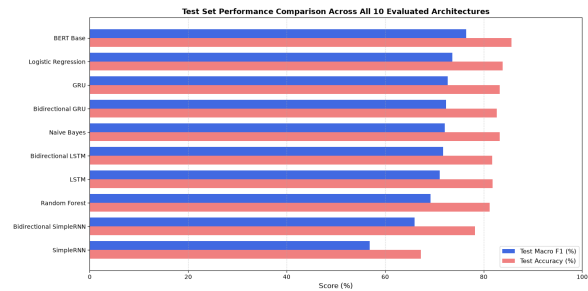


Figure 8: Test Macro F1 and accuracy across all ten architectures.

The accuracy gain is expensive. BERT exceeds Logistic Regression by 2.80 Macro F1 points while requiring $49\times$ the training time and $1,336\times$ the inference time. Logistic Regression classifies all 303,213 test documents in 0.2 seconds; BERT needs 4.5 minutes. Naive Bayes reaches 0.7211—within 4.4 points of BERT—from 0.1 seconds of training. For a routing system operating at CFPB volume, the linear baseline is the stronger engineering choice unless the 2.8-point gain carries specific regulatory value.

Sequence modelling does not pay for itself. Every recurrent architecture is beaten by Logistic Regression. On a task where the discriminative signal is largely lexical—the word clouds in Figure 4 show product vocabulary is highly separable—word order contributes little that bag-of-bigrams does not already capture.

Bidirectionality helps only ungated cells. SimpleRNN gains 9.15 points from bidirectional processing, but LSTM gains only 0.63 and GRU loses 0.40, both within the ± 0.85 -point run-to-run variance we measured for these models. A gated cell already propagates information across the sequence effectively enough that a reverse pass adds nothing, while costing roughly 70% more training time and twice the parameters.

#	Model	Macro F1	Accuracy	Weighted F1	Train (s)	Infer (s)
1	BERT Base †	0.7647	0.8562	0.8613	1678.1	267.3
2	Logistic Regression	0.7367	0.8390	0.8468	34.5	0.2
3	GRU	0.7274	0.8326	0.8420	77.2	5.8
4	Bidirectional GRU	0.7234	0.8267	0.8376	130.6	8.4
5	Naive Bayes	0.7211	0.8329	0.8357	0.1	0.2
6	Bidirectional LSTM	0.7173	0.8175	0.8274	139.1	9.2
7	LSTM	0.7110	0.8180	0.8276	78.2	5.8
8	Random Forest	0.6920	0.8121	0.8197	31.0	0.8
9	Bidirectional SimpleRNN	0.6600	0.7824	0.7944	124.2	8.9
10	SimpleRNN ‡	0.5685	0.6722	0.6943	80.5	5.9

Table 2: Held-out test results for all ten architectures. Every model was trained on the same 200,000-document budget and scored on the same 303,213-document test split. † best-performing; ‡ worst-performing. Inference time is a pure prediction pass over the full test set, reported separately from training.

4.2 Padding Artefacts in Recurrent Models

An earlier iteration of this pipeline read the final hidden state directly off the padded sequence. Because the classical preprocessing path reduces the median narrative to 44 tokens while sequences are padded to 256, that hidden state was advanced through roughly 200 <PAD> steps before being classified. Under that configuration SimpleRNN reached Macro F1 0.2222 and its accuracy fell below the 1/9 random baseline. Masking padded positions with `pack_padded_sequence` raises the same architecture, on the same data and configuration, to **0.6121** validation Macro F1—a $2.8\times$ improvement obtained without changing the model.

This has an interpretive consequence. The gap between SimpleRNN and gated cells narrows from 51.7 points to 10.3 points once padding is masked, so most of the apparent advantage of gating on this task was a padding artefact rather than a property of the recurrence.

4.3 Error Analysis

Figure 9 shows row-normalised confusion matrices. The dominant error is consistent across paradigms and is exactly the one the EDA predicts: **14–17% of Money transfer complaints are predicted as Bank account or service**, the two classes whose word clouds overlap most. *Payday loan* is the hardest class everywhere (recall 0.64–0.67), consistent with it being rarest at 1.13%, while *Mortgage* is easiest (0.93–0.95) on a distinctive vocabulary.

BERT’s advantage is qualitative, not merely quantitative. It nearly halves the *Payday*→*Vehicle loan* confusion (0.05 versus 0.10 for Logistic Regression) but *worsens* *Payday*→*Debt collection* (0.11 versus 0.07). Context helps it separate loan products by their described terms, while making it more willing to assign a complaint describing

collection activity to the collection class.

4.4 Full Per-Class Results

Table 3 gives the complete classification report for one model from each paradigm. Per-class F1 tracks support only loosely. *Mortgage* (5.89% of the corpus) is the second-easiest class everywhere on a distinctive vocabulary of *escrow*, *servicer* and *foreclosure*, while *Credit card / prepaid card*, at 8.10% and therefore better supported, scores 13 F1 points lower. Lexical distinctiveness, not frequency, is what separates the easy classes from the hard ones; the exception is *Payday / title / personal loan*, which is both rarest and lexically closest to two neighbouring loan categories, and is last in every model.

The precision column is where inverse-frequency weighting shows its cost. For the three rarest classes, recall is high (0.64–0.78) while precision is not (0.35–0.72), and F1 penalises the exchange. This is the per-class view of the aggregate effect quantified in Section 4.5.

4.5 Imbalance Handling

Table 4 reports thirteen controlled runs, visualised in Figure 10. The metric of interest is Min-4 F1, the mean F1 over the four rarest classes, since these are what mitigation exists to improve.

Mitigation is usually harmful. In three of the five architectures—Logistic Regression, Naive Bayes and Bidirectional LSTM—the best strategy for minority-class F1 is **no mitigation at all**. Class weighting costs the Bidirectional LSTM 7.9 points and Logistic Regression 4.8 points on precisely the classes it is intended to rescue. The mechanism is a precision/recall trade: inverse-frequency weighting raises minority recall while sharply reducing

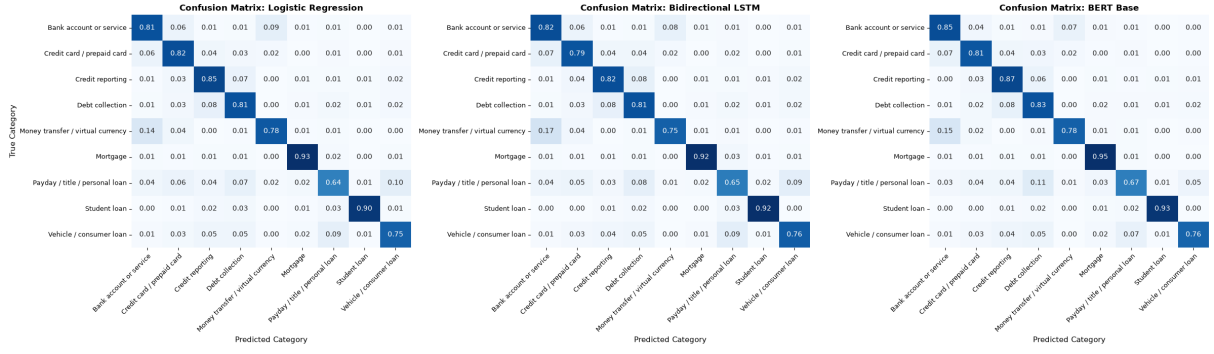


Figure 9: Row-normalised confusion matrices for one model from each paradigm.

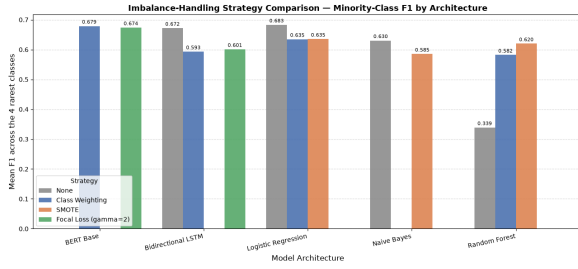


Figure 10: Minority-class F1 by architecture and imbalance strategy.

minority precision, and F1 penalises the exchange. Reporting recall alone, as is common, conceals this.

Mitigation rescues only the model that fails without it. Random Forest is the sole architecture that benefits decisively, gaining 28.1 points of Min-4 F1 from SMOTE and 24.3 from class weighting. Untreated it scores 0.3389, because a depth-limited forest splitting on a 59.6% majority rarely isolates minority regions at all. This suggests reframing imbalance handling as *compensation for an architecture that cannot absorb skew*, rather than as a technique that improves an architecture that can.

The hypothesis of architecture-dependence is not supported. We expected focal loss to win for deep models and SMOTE for linear ones. Neither holds. Focal loss loses to class weighting on BERT (0.6736 versus 0.6789) and to no mitigation on Bi-LSTM (0.6010 versus 0.6718); SMOTE loses on both linear models and wins only on the non-linear Random Forest. Because “no mitigation” wins for both a classical and a deep model, the winning strategies overlap across paradigms and the architecture-conditioned claim must be rejected.

Cost. SMOTE expands the training set from 200,000 to 1,072,881 rows. Its best result (Ran-

dom Forest, 0.6198) is still 6.3 points below what Logistic Regression achieves with no mitigation in 28 seconds. On this corpus, the most elaborate rebalancing applied to the model that needs it most does not reach the simplest model left alone.

5 Conclusion

We compared ten architectures for nine-way CFPB complaint classification under a fixed training budget and a common test split. BERT Base is the most accurate (Macro F1 0.7647) but Logistic Regression retains 96% of that performance at roughly 1/1300 of the inference cost, and every recurrent model is beaten by the linear baseline—evidence that this task is predominantly lexical.

Our main contribution is negative and, we think, useful: on a corpus with a 52.9:1 imbalance ratio, standard imbalance handling degraded minority-class F1 for three of five architectures. It helped decisively only for Random Forest, the one model that collapses without it. We also showed that masking padded positions accounts for most of the apparent superiority of gated cells over simple recurrence here.

Limitations. The 200,000-document budget, while applied uniformly, is 14% of the available training data; absolute scores would rise with more data, though the relative ranking held between validation and test throughout. We did not seed PyTorch initialisation, and measured run-to-run variance of approximately ± 0.85 Macro F1 points on the recurrent models; differences smaller than this—notably the bidirectional effects on GRU and LSTM—should be treated as ties. Focal loss was evaluated at $\gamma = 2$ with inverse-frequency α weighting only, so our comparison isolates the focal modulation on top of class weighting rather than focal loss in isolation. Finally, all findings are

Class	P	R	F1
<i>Logistic Regression</i> — acc. 0.8390			
Credit reporting	0.9686	0.8485	0.9046
Mortgage	0.8489	0.9275	0.8865
Student loan	0.7240	0.8977	0.8015
Bank account or service	0.7749	0.8109	0.7925
Credit card / prepaid card	0.6999	0.8191	0.7548
Debt collection	0.7037	0.8099	0.7531
Money transfer / virtual cur.	0.6607	0.7843	0.7172
Vehicle / consumer loan	0.4556	0.7518	0.5674
Payday / title / personal loan	0.3492	0.6433	0.4527
<i>macro avg</i>	0.6873	0.8103	0.7367
<i>Bidirectional LSTM</i> — acc. 0.8175			
Credit reporting	0.9687	0.8166	0.8862
Mortgage	0.8437	0.9208	0.8806
Bank account or service	0.7532	0.8198	0.7851
Student loan	0.6360	0.9180	0.7514
Credit card / prepaid card	0.6755	0.7949	0.7304
Debt collection	0.6569	0.8051	0.7234
Money transfer / virtual cur.	0.6855	0.7511	0.7168
Vehicle / consumer loan	0.4362	0.7624	0.5549
Payday / title / personal loan	0.3172	0.6541	0.4272
<i>macro avg</i>	0.6637	0.8048	0.7173
<i>BERT Base</i> — acc. 0.8562			
Credit reporting	0.9703	0.8668	0.9156
Mortgage	0.8497	0.9523	0.8981
Bank account or service	0.7867	0.8470	0.8157
Student loan	0.6887	0.9295	0.7912
Debt collection	0.7178	0.8287	0.7693
Credit card / prepaid card	0.7289	0.8113	0.7679
Money transfer / virtual cur.	0.7165	0.7767	0.7454
Vehicle / consumer loan	0.5512	0.7602	0.6391
Payday / title / personal loan	0.4533	0.6681	0.5401
<i>macro avg</i>	0.7181	0.8267	0.7647

Table 3: Full classification report on the held-out test split for one model per paradigm, classes ordered by F1 within each block. Support is identical across blocks and is given in Table 1 scaled to the 15% test partition.

specific to this corpus and label scheme.

Future work. Three directions follow. First, threshold tuning per class is a cheaper lever than reweighting and directly targets the precision/recall trade we identified. Second, the *Money transfer/Bank account* confusion is a label schema question as much as a modelling one and may warrant a merged or hierarchical target. Third, an ensemble of the linear and transformer models—which make qualitatively different errors—should be tested, since their disagreement pattern suggests complementary strengths.

Reproducibility

All results are produced by a staged pipeline with disk-cached artefacts; no metric is hardcoded. The random seed is 42 for all splits and classical models. Experiments ran on an NVIDIA RTX 4070

Model	Strategy	Macro F1	Min-4 F1
Log. Reg.	None	0.7703	0.6829
	Class Weighting	0.7367	0.6347
	SMOTE	0.7348	0.6353
Naive Bayes	None	0.7211	0.6304
	SMOTE	0.6873	0.5853
Rand. Forest	None	0.5702	0.3389
	Class Weighting	0.6920	0.5820
	SMOTE	0.7142	0.6198
Bi-LSTM	None	0.7627	0.6718
	Class Weighting	0.7088	0.5931
	Focal Loss	0.7090	0.6010
BERT Base	Class Weighting	0.7647	0.6789
	Focal Loss	0.7506	0.6736

Table 4: Imbalance-handling comparison. Min-4 F1 is the mean F1 over the four rarest classes. All arms use the identical 200,000-document budget and the identical full test split; only the strategy varies. Naive Bayes exposes no class-weight parameter.

Ti SUPER (16 GB), total wall clock approximately 3.5 hours. Figures were produced with Matplotlib (Hunter, 2007) and NumPy (Harris et al., 2020); SMOTE via imbalanced-learn (Lemaître et al., 2017).

References

- Namig Abbasov. 2024. Consumer complaint dataset. Kaggle. <https://www.kaggle.com/datasets/namigabbasov/consumer-complaint-dataset>.
- Leo Breiman. 2001. Random forests. *Machine Learning*, 45(1):5–32.
- Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar. Association for Computational Linguistics.
- Consumer Financial Protection Bureau. 2024. Consumer complaint database. <https://www.consumerfinance.gov/data-research/consumer-complaints/>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of*

- the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.
- Jeffrey L. Elman. 1990. Finding structure in time. *Cognitive Science*, 14(2):179–211.
- Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al. 2020. Array programming with NumPy. *Nature*, 585(7825):357–362.
- Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- John D. Hunter. 2007. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95.
- Nathalie Japkowicz and Shaju Stephen. 2002. The class imbalance problem: A systematic study. *Intelligent Data Analysis*, 6(5):429–449.
- Thorsten Joachims. 1998. Text categorization with support vector machines: Learning with many relevant features. In *Proceedings of the 10th European Conference on Machine Learning (ECML)*, pages 137–142.
- Justin M. Johnson and Taghi M. Khoshgoftaar. 2019. Survey on deep learning with class imbalance. *Journal of Big Data*, 6(1):1–54.
- Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. 2017. Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17):1–5.
- Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988.
- Andrew McCallum and Kamal Nigam. 1998. A comparison of event models for naive Bayes text classification. In *AAAI-98 Workshop on Learning for Text Categorization*, pages 41–48.
- Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.
- Radim Řehůřek and Petr Sojka. 2010. Software framework for topic modelling with large corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, pages 45–50, Valletta, Malta.
- Mike Schuster and Kuldip K. Paliwal. 1997. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681.
- Karen Spärck Jones. 1972. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics.

A Complete Hyperparameter Tuning Grid

Table 5 records all 30 tuning runs required by the project brief: three configurations for each of the ten architectures, scored on the validation split. The configuration carried forward to the test set is the one maximising validation Macro F1 within each block, shown in bold.

B Inverse-Frequency Class Weights

Table 6 lists the balanced class weights applied in the class-weighting arms of Section 4.5, computed as $n/(k \cdot n_c)$ over the training split. The spread is the quantitative statement of the imbalance problem: a *Payday loan* example contributes 52.9 times the gradient of a *Credit reporting* example, which is both why the technique raises minority recall and why it costs minority precision.

Model	Configuration	Val Acc	Val Macro F1	Val Wtd F1	Train (s)
<i>Classical ML — TF-IDF, 25,000 features</i>					
Logistic Regression	$C=0.1$, balanced, lbfgs	0.8229	0.7220	0.8321	19.52
	$C=1.0$, balanced, lbfgs	0.8375	0.7346	0.8454	37.59
	$C=5.0$, balanced, lbfgs	0.8409	0.7323	0.8477	75.69
Naive Bayes	$\alpha=0.01$	0.8313	0.7195	0.8340	0.37
	$\alpha=0.1$	0.8297	0.7155	0.8324	0.35
	$\alpha=1.0$	0.8190	0.6362	0.8171	0.33
Random Forest	50 trees, depth 20, balanced	0.7806	0.6520	0.7916	5.97
	50 trees, depth 30, balanced	0.7952	0.6701	0.8050	12.16
	50 trees, depth 50, balanced	0.8112	0.6899	0.8188	32.82
<i>Recurrent — Word2Vec init., 256-token sequences, 4 epochs</i>					
SimpleRNN	$h=64$, drop 0.3, lr 10^{-3}	0.7189	0.5412	0.7348	104.00
	$h=128$, drop 0.3, lr 10^{-3}	0.7250	0.5922	0.7446	100.26
	$h=128$, drop 0.4, lr 5×10^{-4}	0.7516	0.6121	0.7667	101.46
GRU	$h=64$, drop 0.3, lr 10^{-3}	0.8093	0.7099	0.8206	93.90
	$h=128$, drop 0.3, lr 10^{-3}	0.8237	0.7153	0.8356	88.32
	$h=128$, drop 0.4, lr 5×10^{-4}	0.8155	0.7093	0.8270	76.61
LSTM	$h=64$, drop 0.3, lr 10^{-3}	0.8111	0.7057	0.8215	72.58
	$h=128$, drop 0.3, lr 10^{-3}	0.8154	0.7098	0.8261	78.21
	$h=128$, drop 0.4, lr 5×10^{-4}	0.8072	0.6955	0.8181	81.06
Bi-SimpleRNN	$h=64$, drop 0.3, lr 10^{-3}	0.7853	0.6609	0.7977	137.46
	$h=128$, drop 0.3, lr 10^{-3}	0.7975	0.6748	0.8072	127.33
	$h=128$, drop 0.4, lr 5×10^{-4}	0.7810	0.6644	0.7946	138.05
Bi-GRU	$h=64$, drop 0.3, lr 10^{-3}	0.8239	0.7161	0.8345	149.36
	$h=128$, drop 0.3, lr 10^{-3}	0.8200	0.7201	0.8308	141.84
	$h=128$, drop 0.4, lr 5×10^{-4}	0.8066	0.7049	0.8185	136.54
Bi-LSTM	$h=64$, drop 0.3, lr 10^{-3}	0.8255	0.7178	0.8352	147.67
	$h=128$, drop 0.3, lr 10^{-3}	0.8216	0.7136	0.8321	136.26
	$h=128$, drop 0.4, lr 5×10^{-4}	0.8158	0.7104	0.8266	121.08
<i>Transformer — WordPiece, 256 tokens, 2 epochs, batch 32</i>					
BERT Base	lr 2×10^{-5}	0.8558	0.7647	0.8609	1678.12
	lr 3×10^{-5}	0.8444	0.7480	0.8534	1712.80
	lr 5×10^{-5}	0.8430	0.7499	0.8495	1755.91

Table 5: All 30 hyperparameter tuning runs, evaluated on the 303,213-document validation split. Bold marks the configuration selected per architecture on validation Macro F1 and carried to the test set in Table 2. Note that for Logistic Regression the highest validation *accuracy* ($C=5.0$) is not the highest Macro F1 ($C=1.0$); selection on accuracy would have chosen a model marginally worse on the minority classes.

ID	Class	Weight
0	Bank account or service	1.9475
1	Credit card / prepaid card	1.3720
2	Credit reporting	0.1864
3	Debt collection	0.8418
4	Money transfer / virtual currency	5.2214
5	Mortgage	1.8856
6	Payday / title / personal loan	9.8510
7	Student loan	5.0767
8	Vehicle / consumer loan	5.4073

Table 6: Balanced class weights over the training split, with the integer label IDs used by the confusion matrices in Figure 9.